A Fast Edge Extraction Method for Mobile Lidar Point Clouds

Shaobo Xia and Ruisheng Wang

***Abstract*— Edges in mobile light detection and ranging (lidar) point clouds are important for many applications but usually overlooked. In this letter, we propose a fast edge extraction method for mobile lidar. First, an edge index based on geometric center is introduced and then gradients in unorganized 3-D point clouds are defined. By analyzing the ratio between eigenvalues, edge candidates can be detected. Finally, an edge linking algorithm named graph snapping is proposed. The method is tested extensively and the experimental results demonstrate that the proposed method is able to quickly extract most of 3-D edges with higher accuracy than the existing methods.**

***Index Terms*— Edge extraction, gradient, graph snapping, mobile light detection and ranging (lidar).**

I. Introduction

MOBILE light detection and ranging (lidar) is a powerful technology to obtain dense 3-D points along roads. Object detection and classification from point clouds have become a hot topic in the recent years [1]. However, few research studies focused on low-level feature extraction [2], for example, edge detection.

Edges in lidar point clouds are geometric discontinuities, including boundary element (e.g., roof outlines) and fold edges (e.g., surface intersection lines) [3]. Edge extraction from lidar point clouds is basis for many applications, such as 3-D reconstruction [4], registration [5], object detection [6], road extraction [7], and data simplification [8]. In 2-D images, many classical algorithms (see [9]) have demonstrated their abilities in extracting various edges. However, this is not the case in lidar points that contain complex geometric information and vary in density.

To take advantage of classical 2-D edge detection algorithms, 3-D point clouds are often transformed to 2-D images or voxels. For example, Lin *et al.* [10] project 3-D points to 2-D images from different viewpoints and use LSD algorithm [11] to find straight lines in the images. These lines are then projected back to 3-D and connected by region growing. Xu *et al.* [12] utilize 3-D Sobel detector to find linear curb candidates in voxelized lidar points and connect curbs based on points' local directions. These methods usually loss 3-D information or degrade the extraction accuracy. Therefore, methods that directly handle 3-D points are preferred.

Edges usually have different geometric properties from flat regions. Local curvature is used as an edge indicator in [13] and points with their curvatures higher than given thresholds are selected as edges. This method is suitable for small objects in high-quality point clouds. Other features, such as normals [14], eigenvalues, and their combinations derived from principal component analysis (PCA), are often used in detecting edges. For example, Hackel *et al.* [15] try to extract edges in point clouds via supervised classification using these features, but fail in areas where no training data are available.

Most edges in 3-D can be seen as intersections of surfaces. Borges *et al.* [16] first segment point clouds into planes and then localize edges by calculating intersecting lines between neighboring planes. This method may fail in case of small planes. To solve this problem, Ni *et al.* [3] and Kang *et al.* [8] project neighboring points to a local plane and then identify edge points according to the pattern of angular gaps on planes. Moreover, in the edge linking step, Ni *et al.* [3] estimate edge point's orientations using iterative random sample consensus (RANSAC) line fitting then group them through region growing. These methods can achieve good results, especially in small scenes of high-quality data. However, for large and complex scenes, involved parameters need to be carefully selected. Besides, calculating PCA-based features and fitting planes/lines iteratively are time consuming and also sensitive to noise.

In this letter, we propose a new method for extracting edges in the original unorganized 3-D mobile lidar point clouds. The method consists of two steps, edge detection and linking. In the first step, a new edge detector based on geometric centroid is proposed and edge candidates are detected by analyzing eigenvalues. In the linking step, a graph snapping algorithm is proposed to obtain smooth edge segments. Finally, the proposed method is evaluated on various scenes and a test in large-scale street scenes is also conducted.

II. Method

A. Edge Detection

1) Gradients of Points: As point density in mobile lidar varies, neighborhood may contain very few points within a fixed radius. Setting a minimum point's number M can ensure a reasonable neighborhood that reduces the impact of point's density. Thus, the neighborhood $\mathcal{N}$ of point P_c is a set that

Manuscript received February 13, 2017; revised April 17, 2017; accepted May 17, 2017. Date of publication June 12, 2017; date of current version July 20, 2017. The work of S. Xia was supported by the China Scholarship Council and the University of Calgary. *(Corresponding author: Ruisheng Wang.)*

S. Xia is with the Department of Geomatics Engineering, University of Calgary, Calgary, AB T2N 1N4, Canada (e-mail: shaobo.xia@ucalgary.ca.)

R. Wang is with the Department of Geomatics Engineering, University of Calgary, Calgary, AB T2N 1N4, Canada, and also with the Beijing Advanced Innovation Center for Imaging Technology, Capital Normal University, Beijing 100048, China (e-mail: ruiswang@ucalgary.ca).

Color versions of one or more of the figures in this letter are available online at http://ieeexplore.ieee.org.

Digital Object Identifier 10.1109/LGRS.2017.2707467

1545-598X © 2017 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

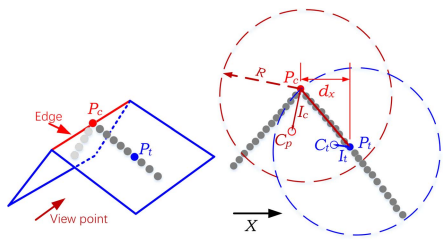


Fig. 1. Two-dimensional example for edge index and gradient along the X-axis for point P_c.

satisfies

$$P_i \in \mathcal{N} \Leftrightarrow \left\| P_c - P_i \right\| \leq R \cup |\mathcal{N}| \geq M \tag{1}$$

in which the default R is 0.15 m and may change adaptively to ensure at least M (i.e., 30) points in the neighborhood of P_c.

A normalized edge index I_c that measures the displacement between current point (e.g., P_c in Fig. 1) and its local geometric centroid (e.g., C_p in Fig. 1) is calculated by

$$I_c = \frac{1}{R}\left\| P_c - \frac{1}{n}\sum_{i=0}^{i=n} P_i \right\| \tag{2}$$

where the subtrahend stands for the geometric centroid and n is the point number within neighborhood. In Fig. 1, R is 1 by default for a clear demonstration. This edge index remains small in nonedge regions, but increases when approaching edge areas and thus reaches local maximum in edges. For instance, a 2-D lateral view of two intersecting planes is given in Fig. 1 in which I_c is much larger than I_t as P_c is an edge point from the intersection line of two planes while P_t is from nonedge area.

Gradients in 2-D images usually indicate the direction along which intensities change most. Analogously, the differences between edge indexes within the neighborhood can be used to define gradients for 3-D point clouds. Just like the gradient decomposition in 2-D images [17], for a given point P_c, 3-D gradient can be decomposed approximately as I_x, I_y, and I_z along three axes by

$$\begin{aligned} I_x &\approx \max_{i\in\mathcal{N}}\left[(I_c - I_i)\cdot\frac{d_{x_{i,c}}}{s_{i,c}}\right] \\ I_y &\approx \max_{i\in\mathcal{N}}\left[(I_c - I_i)\cdot\frac{d_{y_{i,c}}}{s_{i,c}}\right] \\ I_z &\approx \max_{i\in\mathcal{N}}\left[(I_c - I_i)\cdot\frac{d_{z_{i,c}}}{s_{i,c}}\right] \end{aligned} \tag{3}$$

where $s_{i,c}$ is the distance between central point P_c and neighboring point P_i, and $d_{x_{i,c}}$, $d_{y_{i,c}}$, and $d_{z_{i,c}}$ represent the coordinate differences in three axes. A 2-D example for point's gradient along X axis is given in Fig. 1.

2) Edge Candidates: Similar to the change model for a small shift between pixels [18], (4) presents a 3-D shift change $S(x, y, z)$ in point clouds [12], and followed by Taylor series expansion, $S(x, y, z)$ can be expressed as multiplication of a small shift $(\triangle x, \triangle y, \triangle z)$ and a symmetric matrix M in which I_x, I_y, I_z can be calculated based on (3)

$$\begin{aligned} S(x,y,z) &= \sum_{\triangle x,\triangle y,\triangle z}[I_{x+\triangle x,y+\triangle y,z+\triangle z} - I_{x,y,z}]^2 \\ &= \sum_{\triangle x,\triangle y,\triangle z}\begin{bmatrix}\triangle x\cdot I_x + \triangle y\cdot I_y + \triangle z\cdot I_z + \\ O(\triangle x^2, \triangle y^2, \triangle z^2)\end{bmatrix}^2 \\ &\approx (\triangle x, \triangle y, \triangle z)\mathbf{M}(\triangle x, \triangle y, \triangle z)^\top \end{aligned} \tag{4}$$

$$\mathbf{M} = \begin{bmatrix} I_x^2 & I_xI_y & I_xI_z \\ I_yI_x & I_y^2 & I_yI_z \\ I_zI_x & I_zI_y & I_z^2 \end{bmatrix}. \tag{5}$$

The eigenvalues of M capture the distribution of gradients in the local neighbors. A large eigenvalue indicates big changes in gradient along the corresponding eigenvector. This attribute is widely used for feature classification in 2-D images [17]. As for the edges in point clouds, two situations will be observed: 1) intersections of surfaces when two large eigenvalues exist and 2) boundaries of surfaces when only one large eigenvalue exists. Therefore, edge candidates are the points with one or two large eigenvalues. However, it is difficult to define a threshold for "large" eigenvalues. Inspired by [18], instead of using magnitudes of eigenvalues, the ratios between them are used to determine the threshold. Moreover, the time-consuming singular value decomposition (SVD) for calculating eigenvalues will be avoided.

To cancel out the eigenvalues, the first two of sorted eigenvalues ($\lambda_1 \geq \lambda_2 \geq \lambda_3$) are expressed as $\lambda_1 = k_1\lambda_3$ and $\lambda_2 = k_2\lambda_3$ ($k_1 \geq k_2 \geq 1$). Then the ratio between trace and determinant that can be derived from matrix $\mathbf{M}$ is expressed in (6). As the ratio increases with larger k_1 and k_2, we can set a threshold T (e.g., $T = 100$) to guarantee the exist of large eigenvalues

$$\frac{Tr(\mathbf{M})^3}{Det(\mathbf{M})} = \frac{(\lambda_1+\lambda_2+\lambda_3)^3}{\lambda_1\lambda_2\lambda_3} = \frac{(k_1+k_2+1)^3}{k_1k_2} \geq T \tag{6}$$

$$h(c,i) = \frac{1}{\delta\sqrt{2\pi}}e^{-\frac{\|p_c-p_i\|^2}{2\delta^2}}, \quad P_i \in \mathcal{N}. \tag{7}$$

Because gradients in point clouds are discrete and approximate, it is worth noting that the $Det(\mathbf{M})$ will be zero without smoothing. Similar to the methods used in [12], we smooth the $\mathbf{M}$ using Gaussian kernel h (7) where δ is the standard deviation of distance between current p_c and its neighboring point p_i. Besides, to thin the extracted edge points, nonmaximum suppression along gradients is needed. The pseudocode for edge detection is given in Algorithm 1.

Algorithm 1 Edge Detection Based on Gradients

Require: Original point clouds: P_o.
Ensure: Edge points: P.
1: Compute gradients for each point with (3);
2: Smooth I_x, I_y, I_z using (7);
3: Compute (6) and filter points with threshold T;
4: Nonmaximum suppression.
5: **return** Remaining points.

B. Edge Linking

Linking edge points into linear and smooth structures is an essential step in many applications. An edge linking algorithm based on graph snapping is proposed in this section. First, a K-nearest neighbor graph (K-NN) graph G ($k = 10$) based on edge points is constructed. To eliminate noise among edge points, every point P_i is smoothed using (8) where $\mathcal{N}_i$ stands for P_i's neighborhood in the graph. Different from existing orientation estimation methods (e.g., PCA [12] or RANSAC [3]), we select one of the point's connected edges $E_{i,j}$ (i.e., $\overline{P_i} - \overline{P_j}$) as its approximate orientation. For a given point P_i, its orientation $\hat{E}_i$ should satisfy (9), which aims at finding an edge that minimizes the sum of included angles α between itself and other neighboring edges

$$\overline{P_i} = \frac{1}{2}(P_i + \frac{1}{n}\sum_{j=0}^{n} P_j), \qquad j \in \mathcal{N}_i \tag{8}$$

$$\hat{E}_i = \arg\min_{E_{i,j}} \sum_{j,u \in \mathcal{N}_i} \alpha(E_{i,j}, E_{i,u}). \tag{9}$$

After obtaining orientations based on the current graph, if included angles between point's orientation and connected edges are larger than a threshold θ, these edges should be removed. The graph will be updated after all the points have been visited. The aforementioned steps will be repeated for n times with angle threshold θ decreasing by δ_θ in each iteration. Finally, the original K-NN graph will be divided into several subgraphs within which no abrupt orientation changes occur. These steps are termed as graph snapping in this letter (see Algorithm 2 for the complete pseudocode).

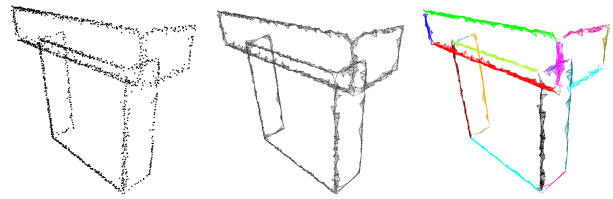
Algorithm 2 Edge Linking by Graph Snapping

Require: Edge points: $P = \{p_1, p_2, ..., p_i, ...\}$.
Ensure: Edge clusters: $E = \{e_1, e_2, ..., e_t ...\}$.
1: Initialize K-NN graph G, $E = \emptyset$;
2: Smooth every point with (8);
3: Estimate orientation for every point with (9);
4: For each point P_i and its edges:
  If $\alpha(E_{i,j}, \hat{E}_i) > \theta$, then remove $E_{i,j}$;
5: $\theta = \theta - \delta_\theta$, repeat Step 2 $\sim$ 5 for n times;
6: Traverse G and push subgraphs e_t into E;
7: **return** E.

An example for graph snapping is given in Fig. 2. Fig. 2(a) shows edge candidates extracted from edge detection step, and then in Fig. 2(b), a K-NN graph is built. After graph snapping, the original graph was separated into several subgraphs that stand for linked edges. In this letter, θ starts from 80° with a decrement 10° and the number of iterations n is set as six for all the experiments.

III. Experiments and Analysis

To evaluate the proposed method, mobile lidar point clouds acquired by Optech Lynx SG1 system along urban street with average density around 1500 points/m^2 were used in tests. All the involved algorithms are implemented in C++ and experiments were conducted on a computer with Intel Core i7-6700HQ 2.6-GHz CPU and 16.0-GB RAM.



(a) (b) (c)

Fig. 2. Example of graph snapping. (a) Edge candidates. (b) K-NN graph ($k = 10$). (c) Subgraphs colored in different colors after snapping.

A. Evaluation and Comparison

We selected five typical samples that are summarized in Table I. A comparison between Ni's algorithms [3] and ours was also conducted. In Fig. 3, original point clouds, edges from Ni's method [3], and our results are listed in the first three rows. Extracted edges are colored randomly and overlap the original point clouds. Line fitting based on our results is performed in the last row for better visualization.

To make a quantitative analysis, three indicators, precision (TP/TP + FP), recall (TP/TP + FN), and F_1 ($2 \cdot$ Precision $\cdot$ Recall/Precision + Recall), are introduced. True positive (TP) indicates the number of correctly detected edges, false positive (FP) indicates the number of wrongly detected edges, and false negative (FN) is the number of undetected edges. F_1 gives a balanced measure over precision and recall. Table I summarizes the evaluation results. Based on the edge definition, manually counted edges were used as ground truth.

In general, the proposed method achieves better accuracy than the method using RANSAC and region growing with much less running time. Compared with Ni's method [3], ours is obviously higher in recall which means more edges can be extracted, such as curb lines in the first column of Fig. 3 and curved boundaries in car and tree samples. Our method is also less sensitive to point density. For example, although the inside of house in Fig. 3(c) has a lower point density than the front, our method [Fig. 3(m)] is able to extract most edges without changing parameters while the contrastive method [Fig. 3(h)] failed. A similar situation can be found in the high building sample. In the last column of Fig. 3, we tested methods on vegetation and pole scene. Although our method showed its ability in extracting edges for rough trunks, low precisions were achieved by both methods due to the false edges detected in the crown.

The reason that the proposed method performs better is in the following. It is difficult to find a suitable neighboring size and fitting error for local planes as point density and noise vary. If a neighborhood contains multiple edges, there will be continuous angular changes within this local plane, and thus edges will be missed, for example, missed edges in building facade in Fig. 3(h) and (i). Besides, RANSAC fitting may need much time if the point clouds were irregular. For example, it takes the method [3] nearly 2000 s to finish

TABLE I
ACCURACY OF EDGE EXTRACTION WITH TWO METHODS

Dataset	Points	Size: $L \times W \times H$ (m^3)	Description	Precision (%)		Recall (%)		F_1(%)		Time (s)	
				Ni's[3]	Ours	Ni's[3]	Ours	Ni's[3]	Ours	Ni's[3]	Ours
Sample1	220,208	$34 \times 2.5 \times 0.9$	Curb	**85.9**	82.5	91.0	**96.4**	88.4	**88.9**	680.6	**16.3**
Sample2	36,963	$6 \times 2.3 \times 2.0$	Car	**89.4**	88.3	92.1	**93.8**	90.8	**91.0**	207.4	**2.9**
Sample3	149,165	$7.8 \times 7.4 \times 11$	Single house	86.4	**87.7**	85.5	**93.9**	86.0	**90.7**	963.6	**11.2**
Sample4	540,964	$39 \times 23 \times 48$	High building	**86.1**	85.0	85.3	**88.4**	85.7	**86.7**	3989.4	**28.7**
Sample5	72,065	$6.6 \times 5.8 \times 11$	Tree & Pole	7.2	**7.6**	81.4	**93.7**	13.2	**14.1**	1979.5	**5.8**

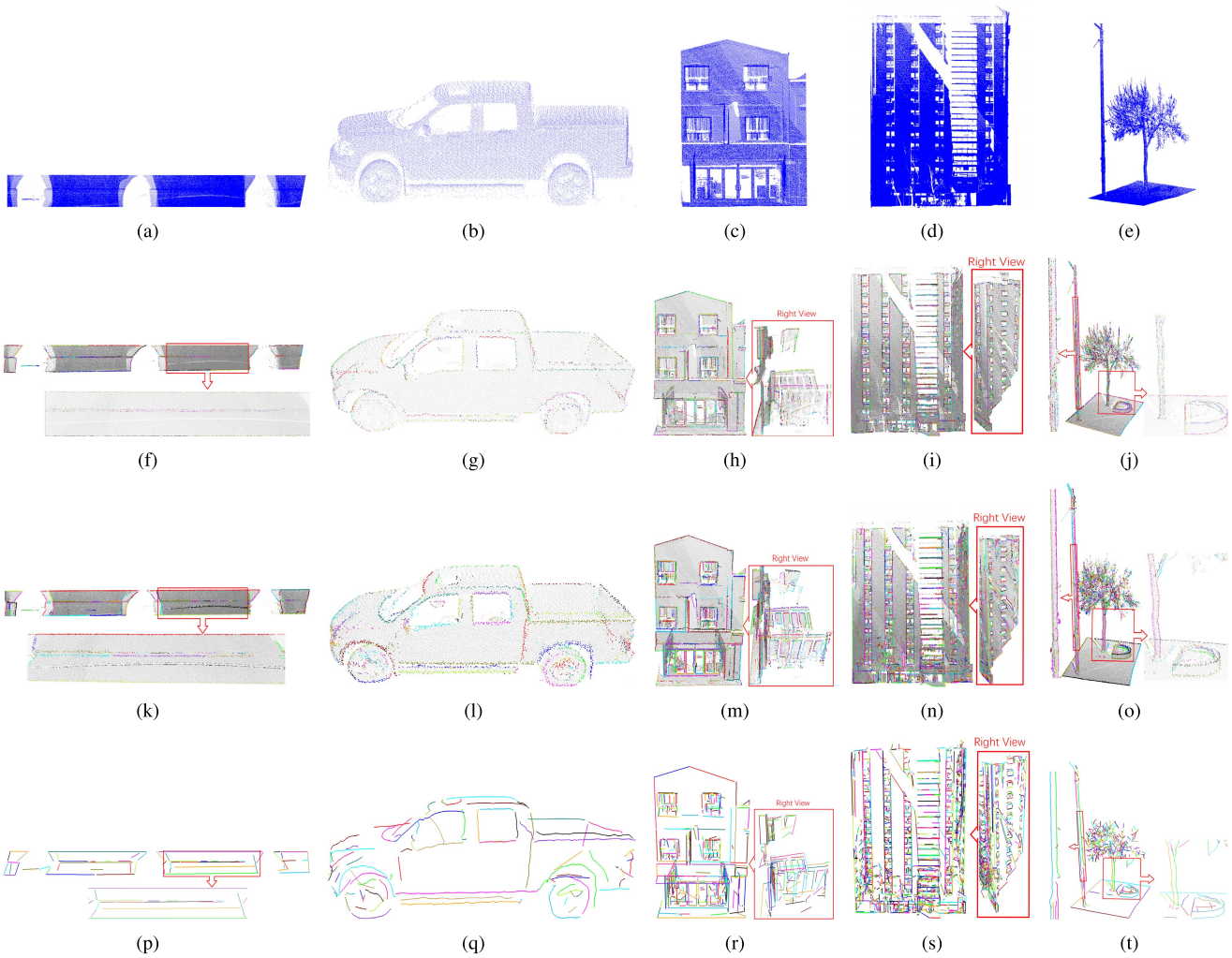
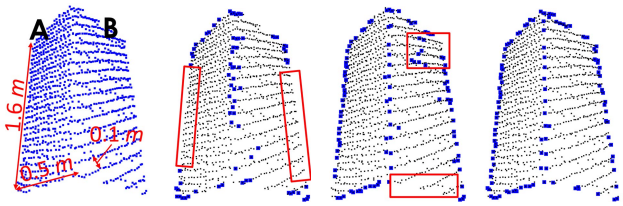


Fig. 3. Experiments of five samples: curb, car, single house, high building, and tree and poles. (a)–(e) Original data. (f)–(j) Results of Ni's method [3]. (k)–(o) Results of the proposed method. (p)–(t) Fitted lines based on extracted edges by our method.

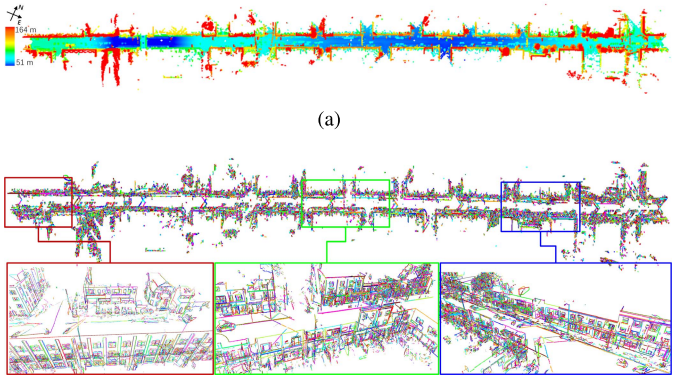
edge extraction, while only 5.8 s is used with our method for Sample 5. In the region growing phase, it is also difficult to determine criterion (e.g., growing angles) for large-scale scenes. Moreover, region growing method suffers from gaps between points [3], and thus leads to fragmentary edges [12]. In contrast, our method is better in overcoming gaps between edge points and likely to produce longer ones. The mean length of extracted edges with Ni's method [3] is 0.67 m and it increases to 0.82 m with our method. A higher percentage (47%) of long edges (over 0.5 m) is also achieved by our method when compared with the existing method (39%) in the first four samples. Furthermore, another advantage of the proposed graph snapping algorithm lies in retrieving topological information between edges, because our method starts from a dense neighboring graph in which all topology information between edges is contained.

It is important to mention that R and M are empirically selected. There are two criteria in selecting radius size R and minimal points M in (1). The first criterion is to consider the size of surface that contains edges. If an edge lies on a small surface [e.g., plane A and B in Fig. 4 (a)], a large R may make the nonedge points yield a large edge index close to the index



(a) (b) (c) (d)

Fig. 4. Example of choosing R and M. (a) Points of intersected plane A and B. (b)$R = 0.5$ m, $M = 30$. (c) $R = 0.1$ m, $M = 0$. (d)$R = 0.1$ m, $M = 30$. Edge points are colored blue and nonedge points are black in (b)–(d).



(a)

(b)

Fig. 5. Results for large-scale tests. (a) Original point clouds. (b) Overview and details of extracted edges.

values of true edge points and thus lead to small gradients in true edge regions. Therefore, many edges will be missed when a large R is used [see Fig. 4(b)]. The second criterion is to consider the gaps between neighboring scan lines. For instance, scan-line gaps in plane B in Fig. 4 are about 0.1 m in width. If these gaps are close or larger than R, most of neighboring points may come from the same scan line, which will result an unexpected edge index and then detected edges will be incomplete and noisy [see Fig. 4(c)]. An acceptable solution is introducing minimal neighbor size M to decrease the influence of gaps. For example, if the neighborhood is constrained by both R and M, the edges can be well extracted as shown in Fig. 4(d).

B. Large-Scale Test

The proposed framework was also tested with a large-scale data set that contains approximately 140 million points and covers a distance of 1.2 km with the elevation difference around 110 m. The original point clouds were manually divided into 17 regions and each contains approximately 8M points. Edges were extracted separately and then combined into one scene. It took about 3.7 h to finish the experiments. Fig. 5(a) shows the original point clouds that are colored according to height. The final edge extraction results are given in Fig. 5(b) with some enlarged details. When examining details in Fig. 5, we can find that the proposed framework can detect and link most of edges correctly, such as facades, windows, vehicles, curbs, and poles.

IV. Conclusion

In this letter, we propose an efficient method for extracting edges from Mobile lidar point clouds. The experimental results demonstrate that our method can achieve better results than the existing method with much less time. The main contributions lies in: 1) introducing gradients into unorganized 3-D point clouds and thus ideas in 2-D image processing can be adapted to 3-D point clouds and 2) a new edge linking algorithm based on graph snapping. Moreover, our method also provides a new tactic in processing point clouds besides RANSAC, PCA, or region growing methods.

References

[1] H. Guan, J. Li, S. Cao, and Y. Yu, "Use of mobile lidar in road information inventory: A review," *Int. J. Image Data Fusion*, vol. 7, no. 3, pp. 219–242, Jun. 2016.

[2] Y. Guo, M. Bennamoun, F. Sohel, M. Lu, and J. Wan, "3D object recognition in cluttered scenes with local surface features: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 36, no. 11, pp. 2270–2287, Nov. 2014.

[3] H. Ni, X. Lin, X. Ning, and J. Zhang, "Edge detection and feature line tracing in 3D-point clouds by analyzing geometric properties of neighborhoods," *Remote Sens.*, vol. 8, no. 9, p. 710, Sep. 2016.

[4] C. Poullis, "A framework for automatic modeling from point cloud data," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 35, no. 11, pp. 2563–2575, Nov. 2013.

[5] T. Koch, M. Korner, and F. Fraundorfer, "Automatic alignment of indoor and outdoor building models using 3D line segments," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. Workshops*, Jun. 2016, pp. 10–18.

[6] R. Wang, F. P. Ferrie, and J. Macfarlane, "A method for detecting windows from mobile lidar data," *Photogramm. Eng. Remote Sens.*, vol. 78, no. 11, pp. 1129–1140, 2012.

[7] D. Zai *et al.*, "3D road surface extraction from mobile laser scanning point clouds," in *Proc. IEEE Int. Geosci. Remote Sens. Symp. (IGARSS)*, Jul. 2016, pp. 1595–1598.

[8] Z. Kang, R. Zhong, A. Wu, Z. Shi, and Z. Luo, "An efficient planar feature fitting method using point cloud simplification and threshold-independent BaySAC," *IEEE Geosci. Remote Sens. Lett.*, vol. 13, no. 12, pp. 1842–1846, Dec. 2016.

[9] J. Canny, "A computational approach to edge detection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 8, no. 6, pp. 679–698, Nov. 1986.

[10] Y. Lin *et al.*, "Line segment extraction for large scale unorganized point clouds," *ISPRS J. Photogramm. Remote Sens.*, vol. 102, pp. 172–183, Apr. 2015.

[11] R. G. von Gioi, J. Jakubowicz, J.-M. Morel, and G. Randall, "LSD: A fast line segment detector with a false detection control," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 32, no. 4, pp. 722–732, Apr. 2010.

[12] S. Xu, R. Wang, and H. Zheng, "Road curb extraction from mobile lidar point clouds," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 2, pp. 996–1009, Feb. 2017.

[13] B. Yang and Y. Zang, "Automated registration of dense terrestrial laser-scanning point clouds using curves," *ISPRS J. Photogramm. Remote Sens.*, vol. 95, pp. 109–121, Sep. 2014.

[14] K. Demarsin, D. Vanderstraeten, T. Volodine, and D. Roose, "Detection of closed sharp edges in point clouds using normal estimation and graph theory," *Comput.-Aided Design*, vol. 39, no. 4, pp. 276–283, Apr. 2007.

[15] T. Hackel, J. D. Wegner, and K. Schindler, "Contour detection in unstructured 3D point clouds," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 1610–1618.

[16] P. Borges, R. Zlot, M. Bosse, S. Nuske, and A. Tews, "Vision-based localization using an edge map extracted from 3d laser range data," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2010, pp. 4902–4909.

[17] C. Harris and M. Stephens, "A combined corner and edge detector," in *Proc. Alvey Vis. Conf.*, 1988, pp. 5210–5244.

[18] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *Int. J. Comput. Vis.*, vol. 60, no. 2, pp. 91–110, 2004.